

上海大学 计算机工程与科学学院

《数据库原理》(1) 实验报告

姓名 赵文宇 学号 23121769

时间 一 3-4 地点 计 706 指导教师 李晓强

实验 5: SQL 命令-视图和交互式 SQL

一、实验目的

- 1、了解和掌握 SQL 命令中视图的建立和修改
- 2、了解和掌握交互式 SQL 使用（包括数据库连接）

二、实验内容

1. lect05 讲稿中的例子练习
2. 对 school 数据库做如下操作：
 - 查询每个学生选课情况（包括没有选修课程的学生）。
 - 检索所有课程都选修的的学生的学号与姓名。
 - 检索选修课程包含 1106 同学所学全部课程的学生学号和姓名。
 - 查询每门课程中分数最高的学生学号和学生姓名。
 - 查询年龄小于本学院平均年龄，所有课程总评成绩都高于所选课程平均总评成绩的学生学号、姓名和平均总评成绩，按年龄排序。
- 3、宿主程序连接数据库；交互式 SQL 使用初探(要求可以使用交互式 SQL 实现数据库的修改)

三、实验过程

（描述实践的过程和结果、遇到的问题 and 解决方案）

- (1) 设计每个工程使用零件总数的视图，并查询零件数大于 50 的工程号

这个例子主要是让我先熟悉视图的创建。我的理解是，视图本质上就是“把一条查询语句包装起来，起一个名字”，这样以后再查的时候就不用每次把复杂的分组统计重新写一遍了（也像是一个“超链接”）。这里先对 SPJ 表按工程号分组，再统计每个工程使用零件的总数，最后基于这个视图继续查询零件数大于 50 的工程号。

具体的 sql 语句如下：

```
--USE gyqk

CREATE VIEW view5_1 as

SELECT JNO, SUM(QTY) as S_num FROM SPJ

GROUP BY JNO;

SELECT JNO FROM view5_1

WHERE S_num > 50;
```

运行结果如下图所示：

	JNO
1	J1
2	J4
3	J5
4	J7

(2) 检索所有课程都选修的学生的学号与姓名

这个题目和上次实验里“查询使用了全部零件的工程名称”在逻辑上非常像，都是一个“全称量词”的问题。SQL 里没有直接的“对于所有”，所以我这里还是用两层 NOT EXISTS 来写。它的含义其实就是：找出这样一些学生，不存在某一门课程，是这个学生没有选修过的。

具体的 sql 语句如下：

```
USE school

SELECT S.xh as 学号, S.xm as 姓名

FROM S

WHERE NOT EXISTS (

    SELECT * FROM C

    WHERE NOT EXISTS (

        SELECT * FROM E

        WHERE E.xh = S.xh AND E.kh = C.kh

    )

)
```

(3) 设计一个视图，包括每个工程项目使用零件总数

像这种按工程汇总后的结果，其实很适合作为一个单独的视图保留下来，后面无论是查询、比较还是做进一步筛选，都会更方便。

具体的 sql 语句如下：

```
--USE gyqk

CREATE VIEW view5_3 as

SELECT JNO, SUM(QTY) as S_NUM

FROM SPJ

GROUP BY JNO

SELECT * FROM view5_3
```

运行结果如下图所示：

结果	消息	
	JNO	S_NUM
1	J1	250
2	J4	300
3	J5	100
4	J6	28
5	J7	300

(4) 设计一个更新限制的上海地区工程视图

这个例子里最重要的点其实不是 SELECT *, 而是最后那个 WITH CHECK OPTION。一开始我只把这个附加语句当成一个“附加条件”，但写完运行了一下，发现它的作用其实是“保证通过视图进行更新之后，这条记录仍然满足视图原本的筛选条件”。

具体的 sql 语句如下：

```
--USE gyqk

CREATE VIEW view_4 AS

SELECT * FROM J

WHERE JCITY = '上海'

WITH CHECK OPTION

SELECT * FROM view_4
```

这个视图只显示上海地区的工程项目，并且由于加了 WITH CHECK OPTION，所以如果以后通过这个视图去更新数据，更新后的结果也必须仍然属于“上海地区工程”这个集合，否则数据库就会拒绝这次更新。

运行结果如下图所示：

结果		消息		
	jno	jname	jcitey	balance
1	J1	东方明珠	上海	0
2	J5	明珠线	上海	123
3	J6	南浦大桥	上海	234
4	J8	地铁两号线	上海	NULL

(5) 判断 V2、V3、V4 上的更新操作是否可行

我先用 sql 语句运行测试了一下，发现不行，后面又按照书上给出的条件判断了一下。

具体的 SQL 语句如下：

```
--USE gyqk;

INSERT INTO view_2(SNAME, SADDR)
VALUES ('马甲公司', '上海小马路 0 号')

-----

UPDATE view_3
SET S_NUM = 100
WHERE JNO = 'J1';

-----

UPDATE view_4
SET JCITY = '南京'
WHERE JNO = 'J1';
```

第一条不可以进行更新，是因为 V2 是由 S、SPJ、P 三张表连接出来的，多表连接视图不允许直接做更新操作。

第二条也不可以进行更新，是因为 V3 里有 GROUP BY 和 SUM()，它不是基本表中的原始行，而是聚合后的结果，所以数据库无法把这类修改再“反推”回原始记录。

第三条还是不可以进行更新，是因为 V4 加了 WITH CHECK OPTION，如果把 JCITY 改成“南京”，那么这条记录更新后就不再属于 V4 视图所定义的“上海地区工程”范围，无法修改。

(1) 查询每个学生选课情况（包括没有选修课程的学生）

这道题比较关键的地方在于“包括没有选修课程的学生”。如果用直接连接，那么没有选课的学生就会被自动过滤掉。我的理解是，以学生表 S 作为主体，把 E 表和 C 表往上“拼接”就可以了，这样即使某个学生没有选课，他的学号和姓名依然会显示出来，只是课程号和课程名这些字段会是 NULL。

具体的 sql 语句如下：

```
--USE gyqk;

INSERT INTO view_2(SNAME, SADDR)
VALUES ('马甲公司', '上海小马路 0 号')
```

```

-----
UPDATE view_3
SET S_NUM = 100
WHERE JNO = 'J1';
-----

UPDATE view_4
SET JCITY = '南京'
WHERE JNO = 'J1';

```

运行结果如下图所示：

结果		消息			
	学号	姓名	课程号	课程名	总评成绩
1	1101	李明	08302001	通信学	NULL
2	1101	李明	08305001	离散数学	60
3	1101	李明	08305002	数据库原理	NULL
4	1101	李明	08305002	数据库原理	NULL
5	1101	李明	08305002	数据库原理	NULL
6	1101	李明	08305003	数据结构	NULL
7	1101	李明	08305004	系统结构	NULL
8	1102	刘晓明	08302001	通信学	NULL
9	1102	刘晓明	08305001	离散数学	87
10	1102	刘晓明	08305002	数据库原理	82
11	1102	刘晓明	08305003	数据结构	NULL
12	1102	刘晓明	08305004	系统结构	74
13	1103	张颖	08302001	通信学	NULL
14	1103	张颖	08305001	离散数学	80
15	1103	张颖	08305001	离散数学	56
16	1103	张颖	08305002	数据库原理	75
17	1103	张颖	08305003	数据结构	84
18	1103	张颖	08305004	系统结构	NULL
19	1104	刘晶晶	08302001	通信学	75
20	1104	刘晶晶	08305001	离散数学	90
21	1104	刘晶晶	08305002	数据库原理	NULL

由于满足条件的记录数量太多，展示篇幅太大，所以只截取了部分运行结果。

(2) 检索所有课程都选修的学生的学号与姓名

这个任务重点考察全称量词的使用。SQL 里没有直接的“对于所有”，所以我这里还是用两层 NOT EXISTS 来写。它的含义其实就是：找出这样一些学生，不存在某一门课程，是这个学生没有选修过的。

具体的 sql 语句如下：

```

USE school

SELECT S.xh as 学号, S.xm as 姓名
FROM S
WHERE NOT EXISTS (
    SELECT * FROM C
    WHERE NOT EXISTS (
        SELECT * FROM E
        WHERE E.xh = S.xh AND E.kh = C.kh
    )
)

```

```
)  
)
```

(3) 检索选修课程包含 1106 同学所学全部课程的学生学号和姓名

这个任务就是上一个任务的变形，只不过“全部课程”不再是课程表 C 中的全部课程，而是“1106 同学已经选过的全部课程”。所以写法依然可以用双重 NOT EXISTS 来处理，逻辑就是：找出不存在某一门 1106 学过、但当前学生没学过的课程的那些学生。

具体的 sql 语句如下：

```
USE school  
  
SELECT S.xh as 学号, S.xm as 姓名  
FROM S  
WHERE NOT EXISTS (  
    SELECT * FROM E as E1  
    WHERE E1.xh = '1106'  
    AND NOT EXISTS (  
        SELECT * FROM E as E2  
        WHERE E2.xh = S.xh  
        AND E2.kh = E1.kh  
    )  
)  
)
```

运行结果如下图所示：

结果	消息	
	学号	姓名
1	1103	张颖
2	1107	张晓峰
3	1102	刘晓明
4	1104	刘晶晶
5	1105	刘成刚
6	1106	李二丽
7	1101	李明

(4) 查询每门课程中分数最高的学生学号和学生姓名

先对每一门课程分别找出其最高总评成绩，再把这个最高分对应的学生找出来。

具体的 sql 语句如下：

```
USE school  
  
SELECT
```

```

E.kh as 课程号, C.km as 课程名, E.xh as 学号, S.xm as 姓名, E.zpcj as 总评成绩
FROM E
JOIN S ON E.xh = S.xh
JOIN C ON E.kh = C.kh
WHERE E.zpcj IS NOT NULL

AND E.zpcj = (

    SELECT MAX(E2.zpcj) FROM E as E2

    WHERE E2.kh = E.kh

)

ORDER BY E.kh, E.xh

```

运行结果如下图所示：

结果	消息				
	课程号	课程名	学号	姓名	总评成绩
1	08302001	通信学	1104	刘晶晶	75
2	08305001	离散数学	1106	李二丽	94
3	08305002	数据库原理	1106	李二丽	90
4	08305003	数据结构	1103	张颖	84
5	08305004	系统结构	1102	刘晓明	74

(5) 查询年龄小于本学院平均年龄，所有课程总评成绩都高于所选课程平均总评成绩的学生学号、姓名和平均总评成绩，按年龄排序

先判断学生年龄是否小于本学院学生的平均年龄,再判断该学生是否存在某一门课，其总评成绩小于等于这门课的平均总评成绩，只要存在这种情况，这个学生就不符合要求。最后对满足条件的学生求个人平均总评成绩，并按年龄从小到大排序即可。

具体的 sql 语句如下：

```

USE school

SELECT

    S.xh as 学号, S.xm as 姓名,

    ROUND(AVG(CAST(E.zpcj AS FLOAT)), 2) as 平均总评成绩

FROM S

JOIN E ON S.xh = E.xh

WHERE E.zpcj IS NOT NULL

AND DATEDIFF(YEAR, S.csrq, GETDATE()) < (

    SELECT AVG(CAST(DATEDIFF(YEAR, S2.csrq, GETDATE()) AS FLOAT)) FROM S as

S2

```

```

        WHERE S2.yxh = S.yxh
    )
    AND NOT EXISTS (
        SELECT * FROM E as E1
        WHERE E1.xh = S.xh
        AND (
            E1.zpcj IS NULL
            OR E1.zpcj <= (
                SELECT AVG(CAST(E2.zpcj AS FLOAT)) FROM E as E2
                WHERE E2.kh = E1.kh
                AND E2.zpcj IS NOT NULL
            )
        )
    )
)
GROUP BY S.xh, S.xm, S.csrq
ORDER BY DATEDIFF(YEAR, S.csrq, GETDATE());

```

3、宿主程序连接数据库；交互式 SQL 使用初探

我编写了一个 python 程序，先连接 school 数据库，再读取课程号为 08305001 的部分记录，接着执行一次更新，最后再查询一次修改后的结果。这样就完成了“宿主程序连接数据库并执行 SQL 修改”的实验要求。

编写的 python 程序如下：

```

import pyodbc

conn = pyodbc.connect(
    "DRIVER={ODBC Driver 17 for SQL Server};"
    "SERVER=localhost;"
    "DATABASE=school;"
    "Trusted_Connection=yes;"
)

cursor = conn.cursor()
kh = '08305001'

print('修改前: ')

cursor.execute("""
    SELECT TOP 5 xh, kh, pscj
    FROM E

```



```

WHERE kh = ?

ORDER BY xh

"", kh)

for row in cursor.fetchall():

    print(row)

cursor.execute("""

    UPDATE E

    SET pscj = pscj + 2

    WHERE kh = ? AND pscj IS NOT NULL

"", kh)

conn.commit()

print('修改后: ')

cursor.execute("""

    SELECT TOP 5 xh, kh, pscj

    FROM E

    WHERE kh = ?

    ORDER BY xh

"", kh)

for row in cursor.fetchall():

    print(row)

cursor.close()

conn.close()

```

这个程序先连接 school 数据库，

```

修改前:
('1101', '08305001', 63)
('1102', '08305001', 90)
('1103', '08305001', 59)
('1103', '08305001', None)
('1104', '08305001', 78)
修改后:
('1101', '08305001', 65)
('1102', '08305001', 92)
('1103', '08305001', 61)
('1103', '08305001', None)
('1104', '08305001', 80)

```

四、收获与感想

这次实验五相比实验四，明显更强调“把查询封装起来”和“把 SQL 放到程序里使用”这两件事。前半部分关于视图的实验，让我感觉视图并不只是“给查

询起个名字”这么简单，它实际上是在数据库里把一段常用逻辑固定下来。像统计工程零件总数、筛选黄色零件供应商、限制上海地区工程这些操作，建成视图后不但查询更方便，而且逻辑也更清晰。尤其是 WITH CHECK OPTION，以前只是记语法，这次真正做了“把上海改成南京”的实验后，才更直观地理解到它是在保证通过视图更新后，数据仍然属于这个视图定义的范围。

school 数据库里的几道题也让我再次体会到 NOT EXISTS 这种写法虽然绕，但在处理“全部满足”“包含全部”这类问题时确实很好用。像“所有课程都选修”“课程集合包含 1106 同学全部课程”“所有课程成绩都高于对应课程平均分”这些题目，本质上都带有一种全称量词的味道，而 SQL 里没有直接的“对所有”，所以最后还是要通过“不存在某个不满足条件的情况”来转化。虽然刚写的时候脑子要拐一下弯，但一旦把逻辑想通，后面写起来其实还挺顺手的。

关于宿主程序连接数据库和交互式 SQL 这个方面的实验操作让我对 SQL 在实际开发中的位置更有感觉了。之前更多是把 SQL 当成单独练习的语句来写，这次开始意识到，在真实系统里它通常都不是“单独存在”的，而是需要通过 Python、Java 或者别的宿主语言去连接数据库、传递参数、接收结果。也就是说，数据库操作并不是写完一条 SQL 就结束了，而是要真正嵌入到程序流程里。